Media Rating System

Develop a backend application for managing media items (e.g., movies or songs) and collecting user reviews. Users can register, browse media, and update ratings and feedback. The system will include JSON API endpoints.

Data structure

The MongoDB database contains three collections. Each rating references one user and one media item. A user or media item can have multiple ratings.

Users

```
_id  ObjectId
username  String
password  String
createdAt  Date
```

Ratings

```
_id  ObjectId
mediaId  ObjectId
userId  ObjectId
rating  Number
comment  String
createdAt  Date
```

Medias

```
_id  ObjectId
title  String
category  String
year  Number
description  String
posterUrl  String
averageRating  Number
```

General requirements

Create the project folder Mediasys_Studentcode.

The resource folder contains sample JSON data, a list of required npm packages, example API endpoints for Postman or Curl, and an EJS template file for rendering views.

Development tools include Visual Studio Code, Postman/Curl, MongoDB Compass, and a web browser.

Backend technologies include Node.js, Express.js, MongoDB, Mongoose, Dotenv, JWT, and CORS.

Setup and registration

Task 1 Project initialization and configuration

Create a database named MyMediaDB. Set up the Users, Medias, and Ratings collections following the provided schema. Import at least a few sample documents into each collection using the provided JSON files or your own test data.

```
Figure 1. Database structure
MyMediaDB
  Medias
  Ratings
  Users
```

Create a .env file defining the server port (PORT), MongoDB connection string (MONGO_URI), and JWT secret (JWT_SECRET). The server must run at http://localhost:1234.

Task 2 POST /users/register

Accept a username and password in the request body. Hash the password using bcrypt before storing it in Users. On success, return a JSON message confirming that the user was created. Return an appropriate failure message if the username already exists or an error occurs.

```
POST http://localhost:1234/users/register
{ "username": "usera", "password": "123456" }
```

Figure 2. Successful registration

```
201 Created
{
  "message": "User registered successfully",
  "user": {
    "_id": "<generated ObjectId>",
    "username": "usera",
    "createdAt": "<creation timestamp>"
  }
}
```

Figure 3. Username already taken

```
409 Conflict
{ "message": "Username already exists" }
```

Browse and rate media

Task 3 GET /medias

Create a public GET endpoint at /medias. Fetch all media items from the Medias collection and return them in JSON format. Include all relevant information for each item, as illustrated in Figure 4.

Figure 4. Media list

GET http://localhost:1234/medias returns 200 OK. One item from the example array is shown below.


```
[
  {
    "_id": "48bbc03134ff4e0189e95c9a",
    "title": "Music Title 8",
    "description": "A sample description for music 8.",
    "type": "music",
    "image": "music8.jpg",
    "createdAt": "2025-07-01T03:10:00.000Z"
  }
]
```


Task 4 PUT /medias/:id/comment

Allow a user to submit a rating and comment for a specific media item. The :id parameter represents the Media ID. Figure 5 shows the request and response.

When the request succeeds:

1. Create a new document in the Ratings collection.

2. Link the rating to the corresponding mediaId.

3. Store the rating score and comment along with the createdAt timestamp.

Rating example and exam rules

Figure 5. Submit a rating and comment

```
PUT http://localhost:1234/medias/c28690cfcb214e90973ced08/comment

{
  "score": 5,
  "comment": "Excellent movie with great storyline!"
}
```

```
201 Created
{
  "mediaId": "c28690cfcb214e90973ced08",
  "userId": "75385c9be8f04013ab5ff89c",
  "score": 5,
  "comment": "Excellent movie with great storyline!",
  "createdAt": "2026-03-15T13:19:15.462Z",
  "_id": "<generated ObjectId>"
}
```

Notes

Students may use different versions of Node.js and MongoDB to complete the project.

Internet access is permitted to install the necessary libraries. Use of other Internet resources, including chat programs, email, remote desktop, and especially AI tools, is considered cheating and will result in a score of zero points.

Students may use offline resources available on their computers.

When compressing the project for submission, exclude the node_modules folder.

Rules

The project will receive zero points if it fails to run or contains runtime errors.